Dada a definição de números naturais a seguir:

```
data Nat = Zero | Suc Nat
   deriving(Eq,Show)
```

implementar as seguintes funções:

(1) Implementar a função `to_int` que converte um `Nat` para um inteiro:

```
to_int :: Nat -> Int
```

(2) Implementar a função `to_nat` que transforma um inteiro em um `Nat`:

```
to_nat ::  Int -> Nat
```

(3) Implementar usando recursão a função `mult` que multiplica dois `Nat`s. (Você **NÃO** pode usar `to_int` ou `to_nat`):

```
mult ::  Nat -> Nat -> Nat
```

(4) Implementar a função `leq` (menor ou igual) que recebe dois `Nat`s e devolve um booleano. (Você **NÃO** pode usar `to_int` ou `to_nat`):

```
leq ::  Nat -> Nat -> Bool
```

(5) Implementar a função `sub` que executa a subtração de dois naturais (Você **NÃO** pode usar `to_int` ou `to_nat`):

```
sub ::  Nat -> Nat -> Nat
```

(6) Implemente recursivamente a função `geq` (maior ou igual) que verifica se o primeiro argumento é maior ou igual do que o segundo (Você **NÃO** pode usar `to_int` ou `to_nat`):

```
geq ::  Nat -> Nat -> Bool
```

(7) A divisão pode ser calculada através de subtrações sucessivas. Implemente a função `divi` usando essa técnica (Você **NÃO** pode usar `to_int` ou `to_nat`):

```
divi ::  Nat -> Nat -> Nat
```

(8) Como você modificaria a função anterior para que ela retorne o resto da divisão?

```
resto_div ::  Nat -> Nat -> Nat
```

(9) Como você implementaria as operações anteriores de forma a elas receberem dois `Int`s e retornarem um `Nat`? Exemplo:

```
mult :: Int -> Int -> Nat
sub  :: Int -> Int -> Nat
divi :: Int -> Int -> Nat
```

(10) Como você implementaria as operações anteriores de forma a elas receberem dois `Nat`s e retornarem um `Int`? Exemplo:

```
mult :: Nat-> Nat -> Int
sub  :: Nat-> Nat -> Int
divi :: Nat-> Nat -> Int
```